

Random Lunch Menu Generator: A Zero-Dependency Web App Scaffold

Student: Polina Ivanilova | **Team:** Individual

Email: pvivanilova@edu.hse.ru | **Date:** 2026-09-14

Assignment: A01 — Random Lunch Generator web app scaffold (Week 1)

Abstract

Choosing a lunch option under time pressure imposes a small but recurring decision cost on daily users. This report presents a zero-dependency web app that randomizes a lunch pick from a fixed menu list, implemented as vanilla HTML/CSS/JavaScript and deployed to GitHub Pages. Functional verification showed that 3 of 12 attempted Font Awesome icons do not exist in the free 6.4.0 icon set, rendering those dishes blank; a switch to native emoji restored 12/12 pictures, and a final upgrade to self-hosted Twemoji SVG artwork (`assets/food/`) provides consistent, offline illustrations for every dish. The key takeaway is that free icon libraries silently omit PRO-only glyphs, so verifying glyph availability against the shipped stylesheet is mandatory before relying on icon markup.

Index Terms — menu generator, random selection, GitHub Pages, Font Awesome, Twemoji, SVG, verification

1. Introduction

Problem statement. Users routinely face "what to eat for lunch?" decisions that consume disproportionate cognitive effort for a low-stakes outcome, a classic instance of decision fatigue described in [4]. A scaffold for a recommender should remove that friction with minimal user interaction — one click, one answer.

Motivation. This is the first assignment of an AI-assisted Recommender Systems course, whose goal is a working baseline that later weeks will extend (content-based filtering, collaborative filtering, two-tower retrieval). A visual, deployable web front end gives a concrete target UI that later recommendation logic can plug into.

Concrete example. The user opens `index.html` (or the live page), sees 12 possible dishes, and clicks "Generate Lunch!"; the app shows "Thinking..." for 500 ms and then displays a random dish with a picture, e.g. 🍜 **Ramen**.

Contributions.

- A fully self-contained random lunch chooser (one HTML file originally, later refactored into `index.html` / `style.css` / `script.js`).
- Public GitHub Pages deployment with an automated Pages build.
- A documented icon-rendering failure (PRO-only Font Awesome classes) and its fix, verified by byte-level inspection of the deployed artifact.

2. Related Work

Prior work consulted:

- [1] Font Awesome, "Font Awesome Free Search," Fonticons, Inc. [Online]. Available: <https://fontawesome.com/search?o=r&m=free>. [Accessed: 2026-09-14].
- [2] MDN Web Docs, "Math.random() — JavaScript," Mozilla Developer Network, 2025. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/random. [Accessed: 2026-09-14].
- [3] GitHub Docs, "About GitHub Pages," GitHub, Inc. [Online]. Available: <https://docs.github.com/en/pages/getting-started-with-github-pages/about-github-pages>. [Accessed: 2026-09-14].
- [4] B. Schwartz, *The Paradox of Choice: Why More Is Less*. New York, NY, USA: HarperCollins, 2004.
- [5] Twitter/X, "Twemoji — Twitter Emoji (SVG), CC-BY 4.0," [Online]. Available: <https://github.com/jdecked/twemoji>. [Accessed: 2026-09-14].

Alternatives considered:

- **Server-side backend API** returning the random pick — rejected: a static page needs no backend, and an API adds latency and hosting cost for zero benefit at scaffold scope.
- **Random image APIs (e.g., Unsplash food photos)** — rejected: external dependency, network latency, and non-deterministic results; an icon/illustration is deterministic and offline-friendly.
- **Microsoft FluentUI Emoji** — evaluated for richer "3D" artwork but rejected: the asset set only covers a partial food list (no Pizza, Sushi, Taco, Sandwich, Spaghetti, or Ramen), so it cannot cover a 12-item menu.
- **Pure Font Awesome icons** — initially chosen, then failed for 3 items (see §5); replaced first by emoji, then by self-hosted Twemoji SVGs.

3. Method

Approach. A single-page app keeps the entire menu and randomization logic client-side. The dish set is a constant array of `{name, img}` pairs pointing to self-hosted Twemoji SVG files in `assets/food/`; clicking the button picks a uniformly random index via `Math.floor(Math.random() * menu.length)` [2] and re-renders the dish area after a short spinner state. No frameworks, no build step.

Pipeline.

Figure 1: 4-step "random lunch pipeline. A uniform index draw from the menu array, a 500 ms spinner mask, then a DOM update injecting an `<img>` from `assets/food/` with a CSS fade-in — all client-side, no network calls.



1. User opens the page (local file or GitHub Pages).
2. `generateRandomLunch()` removes the previous animation class and draws a uniform random index.
3. A spinner and "Thinking..." mask shows for 500 ms.
4. The dish's Twemoji SVG and name are injected into the dish area and revealed with a CSS fade-in.

The entire recommendation logic is the following excerpt from `script.js`:

```
const randomIndex = Math.floor(Math.random() * lunchMenu.length);
const selectedLunch = lunchMenu[randomIndex];
foodIcon.innerHTML = ``;
foodName.textContent = selectedLunch.name;
```

Tools & libraries.

- HTML5 / CSS3 / vanilla JavaScript ES6 — no external runtime framework.
- Twemoji SVG artwork (CC-BY 4.0), self-hosted in `assets/food/`.
- Font Awesome **6.4.0** via cdnjs CDN (used for UI chrome: utensils, spinner, random icon).
- GitHub CLI `gh` **2.32.1**, Git **2.37.0.windows.1**, Python **3.11.9** (verification scripts).

Key design decisions.

- *Why self-hosted Twemoji SVGs over system emoji*: system emoji render differently per operating system, whereas Twemoji SVGs provide one consistent flat-vector style everywhere; they are free (CC-BY 4.0), tiny (1–4 KB per dish), and served from `assets/food/` so the app works fully offline.
- *Why `innerHTML` with a fixed local array*: the image markup is built only from our own constant array — no user input ever reaches the template, so there is no injection surface.
- *Why separate files now*: `style.css` and `script.js` were originally empty placeholders while everything lived in one HTML file; splitting them matches the README's declared structure and makes later weeks' additions (e.g., a recommended-item panel) independently editable.
- *Why a uniform `Math.random` pick over shuffle*: for a single recommendation a uniform draw is unbiased and trivially correct; no repetition-bias logic was needed at scaffold stage.

Configuration.

- Repository: `PollyIva/Lunch-Menu-Generator-project` (public).
- Pages: source branch `main`, path `/`; live at <https://pollyiva.github.io/Lunch-Menu-Generator-project/>.
- Font Awesome loaded from <https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css>.

4. Experiments

Setup. Behavioral test environment was a modern desktop browser (Chrome) against both the local file and the deployed Pages URL. A deterministic static check fetched the free 6.4.0 stylesheet from cdnjs and scanned it with a Python regex for a `:before{content:...}` rule for each of the 12 dish icon classes. Additionally, the deployed `script.js` was downloaded and its raw UTF-8 bytes inspected for the pizza emoji (U+1F355, bytes `F0 9F 8D 95`).

Results. The static scan found **9/12** icon classes defined in the free stylesheet and **3 missing**: `fa-bowl-hot`, `fa-pasta`, `fa-bowl` — exactly the Ramen, Pasta, and Soup items. After replacing all food icons with emoji a manual click-through of all 12 dishes rendered 12/12 visible pictures; the final upgrade to self-hosted Twemoji SVGs kept 12/12 dishes visible, now with consistent flat-vector artwork.

Icon class (dish)	Rule in free 6.4.0 CSS	Rendered?
<code>fa-pizza-slice</code> (Pizza)	present	yes
<code>fa-fish</code> (Sushi)	present	yes
<code>fa-hamburger</code> (Burger)	present	yes
<code>fa-leaf</code> (Salad)	present	yes
<code>fa-utensil-spoon</code> (Tacos)	present	yes
<code>fa-bowl-hot</code> (Ramen)	absent	no
<code>fa-bread-slice</code> (Sandwich)	present	yes

Icon class (dish)	Rule in free 6.4.0 CSS	Rendered?
fa-pasta (Pasta)	absent	no
fa-mortar-pestle (Curry)	present	yes
fa-drumstick-bite (Steak)	present	yes
fa-bowl (Soup)	absent	no
fa-fire (BBQ)	present	yes

Comparison vs. baseline. The Font Awesome baseline failed for 3/12 dishes (25%), with no console error — the blanks were silent. The emoji version reduced failures to 0/12. Neither approach changed selection logic, so the comparison isolates the presentation layer.

Verification. Four independent checks, all repeatable:

- 1. Regex scan of the cdnjs CSS (command line) — confirms exactly three missing classes.
- 2. Byte inspection of the deployed `script.js` (intermediate state) — the literal UTF-8 sequence `F0 9F 8D 95` (pizza emoji, U+1F355) was present, i.e., the emoji fix survived push and Pages rebuild.
- 3. Fetch of the live Pages URL with a cache-buster query (`script.js?v=<ts>`) — served content contains the emoji/asset references, confirming Pages serves the new artifact, not a stale cached copy.
- 4. HTTP HEAD on the deployed `assets/food/pizza.svg` — returns 200 with SVG markup, confirming the final Twemoji artwork is actually shipped and reachable on the live site.

5. Discussion

Failure case. After the first deployment, three dishes — Soup, Pasta, Ramen — displayed **no picture at all**; the icon area was blank while the other nine dishes showed icons correctly. No JavaScript error appeared in the browser console.

Root cause. The three dishes used Font Awesome classes that exist only in the paid *PRO* icon set: `fa-bowl-hot`, `fa-pasta`, and `fa-bowl`. The free 6.4.0 stylesheet defines no `:before` content rule for them, so the `<i>` element renders as an empty box. The bug was selective (9/12 worked), which made it look like a data issue rather than a library-availability one. It is a library-availability issue: the developer (assisted by AI) picked icon names from search results without first checking they were shipped in the *free* CDN build.

Fix + verification. The fix had two stages. (1) *Restore*: replaced all twelve food icons with native Unicode emoji and rendered them via `textContent` instead of `<i>` markup — detection → change → confirmation: regex-scanned the free CSS and confirmed the three classes absent; switched the array to emoji; re-fetched the deployed `script.js` and verified the emoji bytes; manual click-through showed 12/12 pictures. (2) *Polish*: upgraded the artwork to self-hosted **Twemoji SVGs** (`assets/food/`), which render identically across operating systems where system emoji vary; confirmed the deployed `assets/food/pizza.svg` returns HTTP 200 and each dish's `<img>` loads.

Figure 2: Proof of work. Live app deployed at <https://pollyiva.github.io/Lunch-Menu-Generator-project/> with all 12 dishes showing Twemoji illustrations.

1 of 1 questions

Which stylish dish-art solution should I implement?

Select one answer

☐ **FluentUI 3D emoji (Recommended)**
Glossy 3D-style food PNGs from msft's github, MIT license, self-hosted in assets/

☐ **Twemoji SVGs**
Flat vector SVGs from jdecked/twemoji, CC-BY 4.0, small and crisp

☐ **AI-generated custom art**
Use an external AI image generator yourself for a unique custom set

☐ **Keep emoji**
Keep current emoji look, don't change anything

☐ **Type your own answer**
Type your answer...

Dismiss

Submit

What worked.

- Self-hosted Twemoji SVGs are dependency-free, offline-friendly, and render identically across platforms — no icon library, no license tier, no OS font variance.
- The three-file refactor (`index.html` / `style.css` / `script.js`) made each fix a one-line-region change instead of an edit inside a large inline `<style>/<script>` block.
- GitHub Pages deployment was one command; the repo connects cleanly and rebuilds automatically on push.

What surprised me.

- The failure was *silent*: no 404, no console error, just empty space. The free icon search on the Font Awesome site obscures which classes ship in the free build.
- GitHub Pages took noticeably longer (~1 min) to serve the new artifact than the commit timestamp suggested; without the cache-buster check I would have wrongly concluded the push failed.
- Curating artwork was the hardest part of a "trivial" app: FluentUI, a common candidate, turned out to cover only a partial food set — no Pizza, Sushi, or Taco — which forced the Twemoji choice.

Next improvement. Add a repetition guard: track the last served dish and exclude it from the next draw, and persist the daily pick in `localStorage` so a user who reopens the page keeps the choice until a new day. This is a small, testable behavioral change on top of the current random baseline.

6. AI Usage Disclosure

AI tools used. `opencode` (AI coding CLI, model `big-pickle`) for scaffolding, refactoring, deployment, and report drafting; GitHub Actions-free branch deployment via `gh` CLI.

How AI was used.

- *Code generation*: initial single-file `index.html` app; later split into three files.
- *Debugging*: identified the missing-icon root cause by scanning the free FA stylesheet for the three icon classes; also found the FluentUI food-set gap.
- *Docs*: authored `README.md`.
- *Deployment*: connected the local repo to `PollyIva/Lunch-Menu-Generator-project`, resolved a push conflict, enabled Pages, verified live URL.

What I personally verified.

- Re-ran the regex scan of the cdnjs CSS myself — confirmed exactly `fa-pasta`, `fa-bowl-hot`, `fa-bowl` absent (9/12).
- Inspected raw UTF-8 bytes of deployed `script.js` (`F0 9F 8D 95 = U+1F355`) in the intermediate emoji state.
- Fetched the live Pages URL with a timestamp query and confirmed updated content.
- Confirmed the deployed `assets/food/pizza.svg` returns HTTP 200 and valid SVG markup.
- Manually clicked through all 12 dishes in a browser and confirmed 12/12 pictures.

What I trusted without verification.

- Visual appeal of the Twemoji artwork on other operating systems/browsers (checked only on my own browser).
- That Font Awesome's remaining UI chrome (spinner, utensils, dice icon) is free-tier — verified afterwards by the same stylesheet scan described in §4.
- Twemoji's CC-BY 4.0 license terms were taken from the repository's `LICENSE` file without independent legal review.

Session log reference. The attached `session.json` (tool-native export) is the audit trail: it records every prompt, tool call, and file modification for this assignment.

References

- [1] Font Awesome, "Font Awesome Free Search," Fonticons, Inc. [Online]. Available: <https://fontawesome.com/search?o=r&m=free>. [Accessed: 2026-09-14].
- [2] MDN Web Docs, "Math.random() — JavaScript," Mozilla Developer Network, 2025. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/random. [Accessed: 2026-09-14].
- [3] GitHub Docs, "About GitHub Pages," GitHub, Inc. [Online]. Available: <https://docs.github.com/en/pages/getting-started-with-github-pages/about-github-pages>. [Accessed: 2026-09-14].
- [4] B. Schwartz, *The Paradox of Choice: Why More Is Less*. New York, NY, USA: HarperCollins, 2004.
- [5] Twitter/X, "Twemoji — Twitter Emoji (SVG), CC-BY 4.0," GitHub Repository. [Online]. Available: <https://github.com/jdecked/twemoji>. [Accessed: 2026-09-14].